PBAS Frontend System Documentation

Budget Automation System

Postgraduate College for Women, Wah Cantt

Document Type: Functional, workflow, role, and technical documentation

Generated: 28 August 2026

Project: PBAS-Frontend

1. System Overview

PBAS Frontend is a browser-based Budget Automation System built for college budget, payroll, resource request, internal revenue, reporting, user management, notification, audit, and profile workflows. The frontend communicates with a backend REST API using JSON requests and bearer-token authentication.

The system is organized as protected modules. A user logs in, the session token is stored in browser local storage, and the protected area shows only those navigation modules for which the logged-in user has matching permissions.

Main Modules Included

- Login splash screen and authentication screens
- Dashboard
- User Management
- Resource Request Management / Requests & Approvals
- Budget Management
- Payroll Management
- Payroll Estimates Management inside Payroll
- Internal Revenue
- Reports
- Audit Logs
- Notifications
- Settings / Profile Management

1A. Problem Statement

Manual budget and payroll handling in an educational institute creates delays, calculation mistakes, missing records, weak tracking, and difficulty in preparing reports for approval authorities. Requests for resources, government grants, internal revenue records, staff payroll, and annual estimates are usually handled through separate papers, spreadsheets, or informal communication. This makes it hard for the Head of Institute, Admin Officer, Accountant, and Office Assistant to know the current status of approvals, remaining budget, utilized amount, payroll commitments, and audit history.

The Budget Automation System solves this problem by providing a centralized web-based platform where users can perform their assigned financial and administrative tasks according to role-based permissions. The system improves transparency, reduces repeated manual calculation, keeps workflow status visible, and supports decision-making through dashboard analytics and reports.

1B. Project Objectives

- To automate budget creation, categorization, year-wise tracking, and remaining-balance visibility.
- To provide a structured request workflow where requests can be created, submitted, verified, approved, or rejected.
- To support role-based access so each user sees and performs only authorized actions.
- To maintain payroll staff records including BPS/rank, basic pay, allowances, deductions, increment date, and increment rate.
- To calculate payroll estimates automatically using first 4 months salary, next 8 months salary, and increment impact.
- To deduct saved payroll estimates and paid payroll amounts from annual payroll allocation for better financial planning.
- To manage internal revenue in a spreadsheet-like format with automatic totals and closing balances.
- To provide dashboard summaries, charts, status indicators, reports, CSV export, notifications, and audit log visibility.
- To reduce paper-based work and improve traceability of financial decisions.

2. Technology Stack

Area	Technology / Language	Purpose
Frontend language	JavaScript with JSX	Application logic and React component structure.
UI framework	React 19.2	Component-driven frontend rendering.
Build tool	Vite 7.2	Development server, bundling, and production build.
UI component library	Ant Design 6	Forms, tables, cards, modals, drawers, tags, tabs, buttons, date pickers, inputs, upload controls.
Icons	@ant-design/icons	Navigation icons, action icons, splash/login icons, payroll/report icons.
Routing	react-router-dom 7	Browser routes, redirects, protected route wrapper.
State management	Redux Toolkit and React Redux	Authentication state, token/user persistence, login/register/reset actions.
Date handling	dayjs	Budget years, report period labels, internal revenue period keys.
Image crop	react-easy-crop + canvas utility	Profile image cropping and avatar preview.
Styling	CSS	Responsive layouts, dashboards, splash screen, panels, charts, cards.
Testing	Vitest, Testing Library, jsdom	Component and route tests.

3. Authentication and Session Flow

1. User opens `/login`, which displays the responsive splash screen.
2. Clicking the login button opens `/login/form`.
3. User submits email and password. The frontend posts to `POST /api/v1/sessions`.
4. On success, backend returns `user` and `token`. Redux stores both and persists them in `localStorage.authState`.
5. Protected routes validate existing token by calling `GET /api/v1/session`.
6. If validation fails or any API returns 401, the frontend removes local auth state and redirects to login.
7. Logout writes an audit-log request if possible, clears Redux/localStorage auth state, and returns the user to login.

4. User Roles and Permissions

The frontend does not hard-code every role's complete permission list. It checks permissions sent by backend in `user.permissions`. Role names are still visible in user creation and some role-specific workflow rules.

Role	Expected Responsibility in This Frontend	Typical Permissions/Actions
Head of Institute	Final authority for approval workflows and high-level review.	Dashboard, reports/audit visibility depending on backend permissions, approve/reject requests, approve/reject payroll

		cycles, view budget and payroll summaries.
Admin Officer	Administrative verification and operational control.	Verify submitted requests, manage or view users where permitted, view budgets, requests, reports, audit logs depending on permission set.
Accountant	Budget, payroll, revenue, estimates, and financial record maintenance.	Create/update budgets, manage payroll staff and cycles, generate payroll, manage payroll estimates, maintain internal revenue, create financial requests.
Office Assistant	Initial request entry and supporting data submission.	Create and submit requests, attach supporting documents, view allowed request states.

Permission Mapping Used by Navigation

Module	Permission Required
Dashboard	`view_dashboard`
User Management	`view_users`
Resource Request Management	`view_requests`
Budget Management	`view_budgets`
Payroll Management	`view_payroll`
Internal Revenue	`view_budgets`
Reports	`view_reports`
Audit Logs	`view_audit_logs` or `view_reports`
Settings	`manage_profile`
Notifications	`view_notifications`

5. Dashboard Workflow

The Dashboard loads live summary data from `GET /api/v1/dashboard`. It shows top-level budget, revenue, request, payroll, and notification indicators.

- Budget totals show total, remaining, and utilized amounts.
- Internal revenue totals show total, remaining, and utilized revenue values.
- Pending approvals and recent activities are shown as operational signals.
- Recent activity records are shown in a table with colored status tags.
- The budget chart is calculated client-side by converting remaining and utilized values into percentages and SVG circle stroke offsets.

6. User Management Workflow

User Management is used to create, edit, activate, and deactivate system users.

Action	Permission	How It Works

View users	`view_users`	Calls `GET /api/v1/users` and lists all users except the current logged-in user.
Create user	`create_users`	Admin fills username, email, CNIC, father name, designation, department, role, and optional image. Frontend sends `POST /api/v1/users`. Backend sends credentials by email.
Edit user	`update_users`	Existing record is loaded into modal. Submit sends `PUT /api/v1/users/:id`.
Activate/deactivate	`change_user_status`	Confirmation modal appears. Submit sends `PATCH /api/v1/users/:id/activation`.

CNIC input is formatted on the frontend. User images are converted to Base64 data URLs before sending to the backend.

7. Resource Request Management Workflow

Requests use a status workflow: draft, submitted, verified, approved, rejected.

Step	Actor	Frontend Action	Backend Endpoint
Create draft request	Office Assistant / permitted user	Fill title, type, department, requested by, priority, amount, details, attachment, and optional budget link.	`POST /api/v1/requests`
Edit request	Creator/permitted user	Allowed only when backend returns `record.actions.can_edit` and user has `update_requests`.	`PATCH /api/v1/requests/:id`
Submit request	Creator/permitted user	Moves request from draft to submitted.	`PATCH /api/v1/requests/:id/submit`
Verify request	Admin Officer	Requires `verify_requests` and backend action flag.	`PATCH /api/v1/requests/:id/verify`
Approve/reject	Head of Institute	Requires `approve_requests` or HOI role. Resource requests verified by Admin Officer are actionable by HOI.	`PATCH /api/v1/requests/:id/approve` or `reject`
Delete	Permitted user	Allowed while request is editable.	`DELETE /api/v1/requests/:id`

Request types include Government Grant, Internal Revenue, and Payroll. Government Grant requests can link to an existing budget option loaded from `GET /api/v1/budgets/options`.

8. Budget Management Workflow

Budget Management handles budget setup, year ranges, amount tracking, and audit preview.

- Budgets load from `GET /api/v1/budgets`.
- Budget creation uses `POST /api/v1/budgets`.
- Budget updates use `PATCH /api/v1/budgets/:id`.
- Budget deletion uses `DELETE /api/v1/budgets/:id`.
- Each budget has name, start year, end year, amount, type, head, and category.
- Duplicate category/range overlap is checked in frontend before save.

- Previous-year archived budgets cannot be edited or deleted.
- Summary cards calculate total budget, remaining amount, utilized amount, year totals, and filtered totals using `useMemo`.

9. Payroll Management Workflow

Payroll Management covers staff master data, payroll cycles, payment tracking, payslips, annual allocation, archive lock, and estimated payroll budgets.

Staff Master

- Staff records include full name, father name, CNIC, personnel number, post, department, employment type, staff category, BPS, gender, contact, email, bank, increment date, increment rate, active status, pay fields, deductions, and notes.
- Staff save uses `POST /api/v1/payroll_staffs` or `PATCH /api/v1/payroll_staffs/:id`.
- Staff deletion uses `DELETE /api/v1/payroll_staffs/:id`.
- Rank-based deduction is suggested by BPS and basic pay: BPS 20 and above, BPS 17-19, BPS 11-16, BPS 5-10, and BPS 1-4 use different formulas.
- A bulk import button imports visible payroll-sheet staff into Staff Master using the existing create-staff API.

Payroll Cycles

- Payroll cycles load from `GET /api/v1/payrolls`.
- New cycle stores month, year, notes, and annual government allocation.
- Generate cycle calls `PATCH /api/v1/payrolls/:id/generate`.
- Approve and reject cycle use `PATCH /api/v1/payrolls/:id/approve` and `reject`.
- Payment status for each payroll item is updated through `PATCH /api/v1/payroll_run_items/:id`.
- Paid amount is deducted from annual payroll allocation in local annual summary calculations.
- Archived years lock future changes and record surrendered remaining balance.

Payroll Estimates Management

The Estimates Management tab calculates estimated payroll budget from staff master data.

Calculation	Formula
First 4 months salary	`monthly_basic_pay x 4`
Next 8 months salary	`monthly_basic_pay x 8`
Increment impact	`increment_rate x 6`
Total estimated salary	`first 4 months + next 8 months + increment impact`

Saved estimates are stored in browser localStorage under `payrollEstimatedBudgetsV1`. The annual payroll summary subtracts estimated reserved payroll from the remaining annual allocation. This is currently frontend-persistent storage; a backend estimates endpoint would be needed for shared multi-user persistence.

10. Internal Revenue Workflow

Internal Revenue is a spreadsheet-like module for BS, Inter, registration, government dues, project, and other revenue heads.

- Revenue period is stored by month, converted to a `YYYY-MM` period key.

- Cells load from `GET /api/v1/internal_revenue_cells?period=YYYY-MM`.
- Cells save through `POST /api/v1/internal_revenue_cells/bulk_upsert`.
- Formula columns are not directly editable.
- Formula logic calculates BS total, Inter total, Grand Total, Total Expense row, and Closing row on the frontend.
- Edits are buffered in a pending-save map and flushed to backend; localStorage also stores rows for immediate browser persistence.
- Export creates a printable/exportable report layout from selected sections.

11. Reports Module

Reports load summarized backend data from `GET /api/v1/reports`. The user can switch Weekly, Monthly, or Yearly period display using Ant Design segmented control and date picker.

Report Techniques Used

- No external chart library is used.
- Report chart bars are custom HTML/CSS progress bars.
- The width of each chart bar is calculated as `value / maxValue * 100`.
- Tables are Ant Design Table components.
- CSV export is generated in the browser using `Blob`, `URL.createObjectURL`, and a temporary download link.
- Report rows are normalized from backend series arrays using helper functions such as `sumSeries` and `makeRows`.
- Status breakdowns use Ant Design Tag and Progress components.

12. Charts and Analytics Technique

Chart	Technique	Data Source
Dashboard budget donut	Custom SVG circle segments using percentage calculations, stroke offsets, and label coordinates from trigonometry.	`data.charts.budget_overview` from Dashboard API.
Reports bar chart	Custom CSS bar fills. Width is calculated from each module count relative to the largest value.	`request_statuses`, `payroll_statuses`, `budget_types`, notifications summary.
Status breakdown	Tags and progress bars from Ant Design.	Reports API and dashboard API summaries.
Budget/payroll summary cards	Client-side `reduce` calculations and formatted PKR currency display.	Loaded budget/payroll arrays plus local allocation/estimate data.

13. Audit Logs

Audit Logs list system activity from `GET /api/v1/audit_logs`. The UI supports module, action, user, role, date-range, and text filters.

- HOI/admin/superadmin can view all logs when backend allows.
- Other permitted users see logs according to backend response.
- Frontend formats raw actions into readable labels: create, update, delete, approve, reject, submit, verify, generate, login, logout, activation, password, and bulk_upsert.
- Internal revenue spreadsheet changes are summarized by period, changed cell count, and preview values.

14. Notifications

Notifications load from `GET /api/v1/notifications`. Header unread count is loaded inside AppLayout when the user has `view_notifications`.

- Single notification read state uses `PATCH /api/v1/notifications/:id`.
- Mark all read uses `PATCH /api/v1/notifications/mark_all_read`.
- Notifications show title, body, category, read/new tag, and optional link path.

15. Settings and Profile Workflow

Settings lets each user maintain profile details, avatar, and password.

- Profile loads from `GET /api/v1/profile`.
- Profile update uses `PATCH /api/v1/profile`.
- Password change uses `PATCH /api/v1/profile/password`.
- Avatar upload uses a hidden file input, object URL preview, react-easy-crop cropper, canvas crop utility, and Base64 image data sent to backend.

16. API Client and Data Handling

- Default API base URL is `http://localhost:3002`.
- It can be overridden with `VITE_API_BASE_URL`.
- All API requests are JSON and include `Content-Type: application/json`.
- Authenticated calls include `Authorization: Bearer <token>`.
- Response parsing handles JSON and plain text.
- HTML backend error pages are converted into a safer message: “Server error. Please try again.”
- On 401, local auth state is removed and user is redirected to login.
- There is an audit-writing helper in the API client, but `shouldWriteAuditLog()` currently returns false, so automatic client-side audit writing is disabled. Explicit logout audit call still exists.

17. Local Storage Used

Key	Purpose
`authState`	Persists logged-in user and token.
`internalRevenueRowsV4`	Browser copy of internal revenue spreadsheet rows.
`internalRevenueMonth`	Selected internal revenue month.
`payrollAnnualAllocationsV1`	Annual payroll allocation values.
`payrollArchivedYearsV1`	Archived payroll years and surrendered balances.
`payrollSavedPayslipsV1`	Recently generated payroll slips stored in browser.
`payrollEstimatedBudgetsV1`	Saved payroll estimate records.

18. Frontend Validation and UI Patterns

- Ant Design Form rules enforce required fields, email format, and required selections.

- Budget amount input sanitizes digits and decimals.
- Budget duplicate checks prevent overlapping category budgets in the selected year range.
- Modal discard confirmations protect unsaved user, budget, request, payroll, and staff edits.
- Archived payroll years and previous-year budgets are locked from mutation.
- File uploads are converted to data URLs for request attachments and user images.
- Responsive design uses CSS flex, grid, media queries, Ant Design breakpoints, and horizontal table scroll.

19. Module-by-Module Action Summary

Module	Main Actions	Primary Actor
Auth	Login, logout, register by invitation, email verification, forgot/reset password.	All users
Dashboard	View live KPIs, activities, budget chart, revenue and request/payroll signals.	Permitted users
Users	Create user, edit user, activate/deactivate user, preview user image.	Admin/HOI as permitted
Requests	Create, edit, submit, verify, approve, reject, delete, view audit.	Office Assistant, Admin Officer, HOI
Budgets	Create, update, delete, search/filter, audit preview, view totals.	Accountant/Admin as permitted
Payroll	Manage staff, import payroll sheet data, create cycles, generate payroll, approve/reject, record payment, print slip, archive year.	Accountant, HOI
Estimates	Select staff, calculate estimated annual salary, save/remove estimate, deduct from allocation.	Accountant
Internal Revenue	Edit spreadsheet cells, add structured revenue records, calculate totals, save, export/print.	Accountant
Reports	Load reports, filter type, switch period, export CSV, view charts/tables.	HOI/Admin/Accountant as permitted
Audit Logs	Filter and inspect audit trail.	HOI/Admin/report viewers
Notifications	View feed, mark one/all read.	Permitted users
Settings	Edit profile, crop avatar, change password.	Logged-in user

20. Current Limitations / Notes

- The complete authoritative role-permission matrix is controlled by the backend and is not fully visible in this frontend repository.
- Payroll annual allocations, archived payroll years, saved payslips, and payroll estimates are currently stored in browser localStorage in the frontend. For multi-user production use, backend persistence endpoints should be added.
- The newly imported payroll sheet data is based on the readable rows in the supplied image. Some blurred names may require manual correction in Staff Master.
- Reports period controls currently change the displayed period label and export filename; the frontend calls a single reports endpoint without sending period filters.
- Expense Management route currently points to a placeholder page.

21. High-Level Data Flow

1. User authenticates and receives token.
2. Protected route validates token and refreshes user permissions.
3. AppLayout filters navigation using permissions.
4. Module page loads data from its API wrapper.
5. User performs an action through forms, tables, modals, drawers, or popovers.
6. API wrapper sends JSON request to backend with bearer token.
7. On success, page reloads affected data and shows success message.
8. On failure, page shows backend or generic error message.
9. Reports and dashboard transform loaded arrays into cards, tables, progress bars, and custom charts.

22. Future Enhancements

- Add backend database tables and APIs for payroll estimates, annual payroll allocations, saved payslips, and archived payroll years so this data is shared across all users and devices.
- Complete the Expense Management module and connect approved requests with actual expense entry and budget deduction.
- Add full PDF export for reports, payroll estimates, internal revenue sheets, and approval summaries.
- Add advanced charts using a charting library if future requirements need interactive graphs, drill-down analytics, or printable dashboards.
- Add a backend-driven role-permission management screen so administrators can edit permissions without code changes.
- Add Excel/CSV import for payroll staff and internal revenue data to reduce manual data entry.
- Add stronger validation for CNIC, bank account/IBAN, payroll year ranges, increment dates, and duplicate staff records.
- Add notification delivery through email or SMS for request approvals, payroll cycle approval, rejected requests, and password/security events.
- Add backup/restore and audit export features for administrative record keeping.
- Add multi-step approval configuration for departments if the college later needs more approval levels.

23. Important Commands for Development, Usage, and Viva Demonstration

This section lists the main command-line commands a developer or system presenter should know while running, testing, building, version-controlling, and demonstrating the PBAS system. The exact backend/database commands may depend on the backend project configuration, but these are the standard commands normally used for a React/Vite frontend with a REST API backend.

Frontend Project Commands

Command	Purpose	When to Use
`npm install`	Installs frontend dependencies from `package.json`.	Run once after cloning the project or when dependencies change.
`npm run dev`	Starts the Vite development server.	Use during development to open the frontend, usually at `http://localhost:5173`.

`npm run build`	Creates a production build in the `dist` folder.	Use before final submission/deployment to verify the frontend compiles.
`npm run preview`	Runs a local preview of the production build.	Use after build to check the final production version locally.
`npm run lint`	Runs ESLint checks.	Use to find code-quality and syntax-style issues.
`npm test`	Runs the Vitest test suite.	Use to verify existing component and route behavior.

Environment and Configuration Commands

Command	Purpose
`cp .env.example .env`	Creates a local environment file from the example file if needed.
`cat .env.example`	Shows available environment variables.
`VITE_API_BASE_URL=http://localhost:3002 npm run dev`	Starts frontend with a specific backend API URL for local testing.

The frontend API base URL defaults to `http://localhost:3002`. If the backend runs on another port, set `VITE_API_BASE_URL`.

Git Commands

Command	Purpose
`git status`	Shows modified, staged, and untracked files.
`git diff`	Shows unstaged code changes.
`git add .`	Stages all changed files for commit.
`git commit -m "message"`	Creates a commit with a meaningful message.
`git push`	Uploads committed changes to the remote repository.
`git pull`	Downloads latest changes from the remote repository.
`git log --oneline --decorate -5`	Shows recent commits in short form.
`git branch`	Shows local branches and current branch.

Useful Codebase Search Commands

Command	Purpose
`rg "view_payroll" src`	Finds where a permission is used.
`rg "api/v1" src/api`	Shows API endpoints used by the frontend.
`rg "Route path" src/App.jsx`	Shows application routes.
`rg "useState|useMemo|useEffect" src/pages`	Finds React state, calculations, and loading flows.

`ls src/pages`	Lists main page modules.
`ls src/api`	Lists API wrapper files.

Backend and API Demonstration Commands

The backend is not inside this frontend folder, but during viva the examiner may ask how the frontend communicates with backend. These commands can be used when the backend server is running.

Command	Purpose
`curl http://localhost:3002/api/v1/dashboard`	Shows that backend routes are reachable. Protected routes may require token.
`curl -H "Authorization: Bearer TOKEN" http://localhost:3002/api/v1/reports`	Shows authenticated API response for reports.
`curl -H "Authorization: Bearer TOKEN" http://localhost:3002/api/v1/budgets`	Shows budget records returned from backend.
`curl -H "Authorization: Bearer TOKEN" http://localhost:3002/api/v1/payrolls`	Shows payroll cycles and staff data returned from backend.

In these examples, replace `TOKEN` with the login token. The token is stored in browser localStorage under `authState`.

How to Show the Database to Examiner

There are three practical ways to demonstrate database data during viva:

1. **Through the application UI:** Open modules such as User Management, Budget Management, Payroll Management, Internal Revenue, Reports, Audit Logs, and Notifications. This is the best user-facing demonstration because it proves the database is connected to the system workflow.
2. **Through API responses:** Use browser developer tools or `curl` commands to show backend JSON responses. This proves the frontend is not static and is receiving database-backed data from REST APIs.
3. **Through database console/tool:** Open the backend database using the relevant database client and run queries against the main tables.

Common Database Commands for Viva

If the backend uses PostgreSQL, the following commands are commonly used:

Command	Purpose
`psql -U postgres -d DATABASE_NAME`	Open PostgreSQL database console.
`\dt`	List database tables.
`\d users`	Show structure of the users table.
`SELECT * FROM users LIMIT 5;`	Show sample user records.
`SELECT * FROM budgets LIMIT 5;`	Show sample budget records.
`SELECT * FROM requests LIMIT 5;`	Show request workflow records.
`SELECT * FROM payroll_staffs LIMIT 5;`	Show payroll staff master records.

`SELECT * FROM payrolls LIMIT 5;`	Show payroll cycle records.
`SELECT * FROM payroll_run_items LIMIT 5;`	Show generated payroll line items.
`SELECT * FROM internal_revenue_cells LIMIT 10;`	Show saved internal revenue spreadsheet cells.
`SELECT * FROM audit_logs ORDER BY created_at DESC LIMIT 10;`	Show recent audit events.
`SELECT * FROM notifications ORDER BY created_at DESC LIMIT 10;`	Show recent notifications.
`\q`	Exit PostgreSQL console.

If the backend uses MySQL, similar commands are:

Command	Purpose
`mysql -u root -p DATABASE_NAME`	Open MySQL database console.
`SHOW TABLES;`	List database tables.
`DESCRIBE users;`	Show table structure.
`SELECT * FROM users LIMIT 5;`	Show sample data.

Browser Developer Tools Commands and Checks

Action	Purpose
Open DevTools: `F12` or `Ctrl + Shift + I`	Inspect frontend errors, network calls, and localStorage.
Network tab	Show API calls such as dashboard, budgets, requests, payrolls, and reports.
Application tab → Local Storage → `authState`	Show saved login user/token state.
Console: `JSON.parse(localStorage.getItem("authState"))`	Display logged-in user and permissions in browser console.
Console: `localStorage.keys` equivalent via `Object.keys(localStorage)`	Show local storage keys used by the frontend.

Recommended Viva Demonstration Flow Using Commands

1. Start backend server in one terminal.
2. Start frontend using `npm run dev`.
3. Open `http://localhost:5173/login` and log in.
4. Use browser DevTools Network tab to show API calls.
5. Open Dashboard and explain live API-backed summary cards/charts.
6. Open Budget Management and show budget records.
7. Open Requests and show draft → submit → verify → approve workflow.
8. Open Payroll and show Staff Master, Payroll Cycles, and Estimates Management calculations.
9. Open Reports and export CSV to demonstrate report generation.

10. Open database console and run `SELECT` queries for users, budgets, requests, payroll, audit logs, and notifications.
11. Run `npm run build` to show the frontend compiles successfully.